

ShopEasy - Reflection Report

Software Testing Term Project

Student: Melike Aldemir

1. Which bugs did each technique reveal?

Task 1 - Specification-Based Testing:

No bugs found in PriceCalculator. The implementation correctly handles edge cases including zero price, full discount, and full tax.

Task 2 - Structural Testing:

ShoppingCart achieved 90% branch coverage. All major branches covered including addItem (new/existing), removeItem (found/not found), updateQuantity, and empty cart.

Task 3 - Design by Contract:

Assertions caught invalid inputs: negative basePrice, discountRate > 100, taxRate > 100, null product, zero/negative quantity.

Task 4 - Property-Based Testing:

The cartCommutativity test revealed that products with the same ID but different prices could cause inconsistency. Fixed by ensuring products are distinct using Assume.that().

Task 5 - Mocks & Stubs:

No bugs found. OrderProcessor correctly handles happy path, inventory failures (payment never called), payment failures, and multiple items with partial availability.

2. Most effective technique per unit of effort

Property-Based Testing (Task 4) was the most effective. With only 4 property definitions, thousands of random combinations were tested automatically. The `cartCommutativity` property caught an edge case that traditional specification-based testing would likely miss. `jqwik`'s automatic shrinking made debugging very easy. This technique provided the best bug-finding power for the least amount of code written.

3. Testing pyramid classification

Unit Tests: 42 tests (100%)

Integration Tests: 0

End-to-End Tests: 0

What's missing: Integration tests between `OrderProcessor` and real `InventoryService`/`PaymentGateway` implementations, performance tests for large numbers, and concurrency tests for `ShoppingCart` (current implementation is not thread-safe).

4. If I had one more week, what would I test next?

Priority 1 - Integration Tests: Test `OrderProcessor` with real (not mocked) dependencies to verify contracts.

Priority 2 - Mutation Testing: Run PIT to find surviving mutants and improve test quality.

Priority 3 - Performance Tests: Ensure `PriceCalculator` handles very large numbers ($1e15$) without overflow.

Priority 4 - Concurrency Tests: Test ShoppingCart with multiple threads adding/removing items simultaneously.